

ANCHOR: Mitigating GPU Shallow Disruptions with Decoupled Memory

Haoyi Ma*, Shiwei Gao*, Youmin Chen†, Junrong Huang, Youyou Lu, Jiwu Shu‡
Tsinghua University †Shanghai Jiao Tong University

Abstract

Job failures are frequent in large-scale GPU clusters for LLM workloads, leading to significant resource wastage. The vast majority of these are shallow disruptions (e.g., software errors or updates), where only the worker process crashes while the underlying GPU and OS kernel remain intact. Existing recovery systems, however, are failure-agnostic, relying on heavyweight checkpointing that forces a full, slow state reload even when the data could have survived on the GPU.

In this paper, we propose ANCHOR, a system that avoids this overhead by decoupling memory ownership from the failure-prone worker process. ANCHOR uses a daemon process to manage GPU memory. Leveraging the GPU's native IPC memory mechanism, ANCHOR enables a restarted worker to remap and reuse the daemon-managed memory with no overhead on the normal execution path. To make this idea practical for real-world ML applications, ANCHOR introduces an access-path-based identity resolution mechanism to remap logical tensors to their physical memory objects after restart. ANCHOR also provides lightweight, application-aware consistency protocols to ensure data integrity with little overhead. We integrate ANCHOR into DeepSpeed and vLLM. Our evaluation demonstrates recovery acceleration, with recovery time reduced by 60.7% for inference and 26.6% for training on average. For Qwen3-235B inference, ANCHOR reduces the recovery time from 143.78 s to 27.85 s.

CCS Concepts: • **Computer systems organization → Reliability**.

Keywords: LLM, GPU memory management, fault tolerance

ACM Reference Format:
Haoyi Ma, Shiwei Gao, Youmin Chen, Junrong Huang, Youyou Lu, and Jiwu Shu. 2026. ANCHOR: Mitigating GPU Shallow Disruptions with Decoupled Memory. In *Symposium on Operating Systems Principles (SOSP '26), September 29–October 2, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3830418.3843892

*Haoyi Ma and Shiwei Gao contributed equally to this work.
‡Jiwu Shu is the corresponding author (shujw@tsinghua.edu.cn).



This work is licensed under a Creative Commons Attribution 4.0 International License.
SOSP '26, September 29–October 2, 2026, Prague, Czech Republic
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2585-2/26/09
https://doi.org/10.1145/3830418.3843892

1 Introduction

Large language models (LLMs) are increasingly used across a wide range of applications. From Llama [4] and DeepSeek [6] to the Gemini [1] family, model parameters have grown from billions to hundreds of billions and even trillions. The GPU resources required for training and serving large models have expanded rapidly alongside their popularity. Leading AI companies commonly employ clusters containing tens of thousands or even hundreds of thousands of GPUs to support large-model training and inference workloads.

In such large-scale GPU clusters, job failures frequently cause tasks to exit prematurely. We analyze publicly available failure traces from three production clusters: Alibaba [8], ByteDance [32], and Shanghai AI Lab [9]. We find a consistent pattern: truly *destructive failures*, where GPU hardware damage or OS kernel panic takes a node offline, account for only a minority of cases. The vast majority of failures are relatively benign: the GPU driver/hardware and OS kernel remain intact while only the job process crashes. We term this failure category *shallow disruption*. The specific causes of shallow disruptions include: (1) software errors (e.g., NCCL timeouts, uncaught exceptions), and (2) user-initiated restarts (e.g., modifying code or data for debugging). Across the three traces, shallow disruptions account for 65.8%–98.8% of all interruptions (§2.1).

Restarting an interrupted job could incur severe resource wastage and loss of valuable compute cycles. To mitigate this, systems typically rely on periodic checkpointing [16, 27, 33], allowing failed jobs to resume by rolling back to the latest valid checkpoint. However, LLM training and inference are memory-consuming; the critical state that must be checkpointed, such as model weights, optimizer states, and KV caches, is large in volume. For example, restarting the inference engine for the Qwen3-235B model takes 143.78 seconds even when the model weights reside in the CPU page cache.

A key observation of this paper is that existing checkpointing mechanisms are intrinsically *failure-agnostic*. They treat all disruptions uniformly, blindly checkpointing and reloading states as if every failure were catastrophic. While this heavy machinery is unavoidable for destructive failures, applying it to shallow disruptions is overkill, making the recovery process unnecessarily expensive. For shallow disruptions, the OS kernel and GPU hardware remain entirely unscathed. To bypass the substantial overhead of node reallocation and environment re-initialization, cluster schedulers typically restart the failed job in-place [8] on the exact same

node. However, modern OS semantics tightly couple memory ownership with the process lifecycle; the crash of the worker process forces the OS to immediately reclaim and discard memory states. This creates a glaring inefficiency in current recovery paradigms: *it is the same GPU, requiring the same data, yet managed by a new process.* This fundamental mismatch leads to our core insight: *We should decouple the memory from the failure-prone worker process, anchor the data on the GPU, and reuse it across shallow disruptions.*

We propose Anchor, which decouples memory ownership from the executing process. Anchor uses a daemon process as a sidecar of the existing ML worker process. The daemon allocates and owns the data; the worker process can access the data, but the data is not destroyed when the worker exits. Moreover, the daemon does not execute any user-defined CPU logic or GPU kernels, and therefore is not subject to unexpected errors. The key enabler to realize Anchor is the IPC memory mechanism on modern GPUs, which provides two key properties. ❶ *Reference counting*: IPC memory maintains a reference count for the memory region; when the worker process exits, the reference count is decremented, but the daemon still owns the data. A newly restarted worker can re-establish communication with the daemon and remap the GPU memory from the previous execution via IPC. ❷ *Virtual-physical address mapping*: IPC memory is implemented through the GPU's hardware page table mapping, allowing the worker process to access the mapped memory seamlessly as if it were self-allocated. In this way, it incurs no extra overhead on the normal execution path. Anchor forwards the worker's requests to the daemon, which allocates the memory and then grants read-write permissions to the client via the IPC mechanism. Anchor still faces two challenges when deployed in real ML systems.

C1: Identity resolution. What the daemon manages are raw GPU memory objects, whereas ML frameworks operate on tensors with application-level semantics. When a new worker starts, Anchor must re-establish the mapping between semantically meaningful tensors and the memory objects retained by the daemon. Since users may modify their code after a disruption (e.g., adding or removing buffers), a naive sequential matching strategy is unreliable. This requires us to provide an identifier for each tensor.

Since Python objects exhibit a graph structure, where variables point to multiple sub-attributes (attributes, array elements, etc.), all memory objects can be reached through top-down traversal. We use the access path (e.g., *engine.kv_cache ['layer_0']*) from the root to each memory object as its identifier. To obtain the access path, Anchor performs a deterministic top-down traversal starting from variables in the global namespace. We use a BFS algorithm, traversing in lexicographic order within each level. In the presence of aliases, Anchor uses the access path that first reaches the object as its identifier.

C2: Consistency. The worker can freely read and write daemon-managed memory during normal execution. If the worker crashes while executing a GPU kernel, the memory in the daemon may be in a partially modified state. To guarantee memory consistency after restart, Anchor provides a general-purpose copy-on-write mechanism that backs up daemon-managed data before writes, at the cost of significant GPU memory overhead. To reduce this cost, we also design lightweight consistency protocols tailored to application semantics.

For training, we observe that each update depends solely on gradients, aside from the weights and optimizer states. The gradients can therefore serve as a *redo log*. Anchor reserves only a single-chunk *undo buffer* for the state tensor currently being updated and records update progress through a persistent cursor. Upon recovery, the system rolls back the in-progress chunk and replays the remaining updates from the gradients retained in the daemon.

For inference, we exploit the append-only property of KV cache writes under PagedAttention: the engine directly overwrites the KV slot for the current token without depending on the old value. Anchor only persists lightweight page table metadata on the CPU side; upon recovery, it rolls back to the last committed position and re-executes the interrupted forward pass, overwriting any partially written slots with correct data.

The main contributions of this paper are as follows:

- For shallow disruptions, we propose the principle of decoupling memory ownership. Through the GPU IPC mechanism, we separate the ownership of memory objects from the executing process and delegate it to a daemon. This allows data to survive shallow disruptions, avoiding the overhead of checkpoint saving and loading (§ 3.2).
- We propose an access-path-based identity resolution mechanism that leverages Python object graph traversal to generate cross-restart stable identifiers for tensors, enabling transparent remapping from logical tensors to physical objects (§ 4.2).
- We design lightweight consistency protocols that ensure daemon-managed memory remains in a consistent state with low overhead (§ 4.3).
- We implement Anchor on the representative training and inference systems DeepSpeed and vLLM. Our evaluation demonstrates recovery acceleration, reducing time by 60.7% for inference and 26.6% for training on average. For large-scale Qwen3-235B inference, Anchor reduces the recovery time from 143.78 s to 27.85 s.

2 Background and Motivation

2.1 Job Disruptions in GPU Clusters

Production GPU clusters now execute LLM jobs that span thousands to tens of thousands of GPUs; at this scale, frequent failures are an expected operating condition [11, 32].

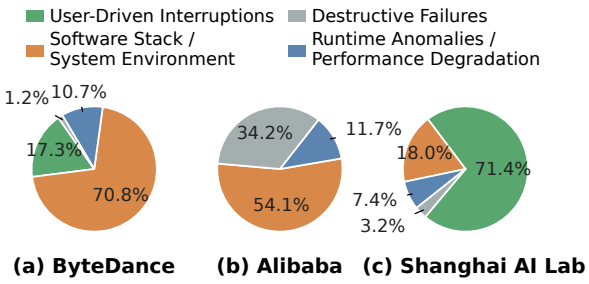


Figure 1. Interruption statistics from three large-scale GPU-cluster studies. Shallow disruptions (software errors, user-initiated restarts, etc.) dominate across all three traces.

We analyze publicly available failure reports from three production clusters, Alibaba [8], ByteDance [32], and Shanghai AI Lab [9], and organize them into four categories:

- *User-Driven Interruptions.* Users restart workers to tune hyperparameters or modify data pipelines and training or inference logic.
- *Runtime Anomalies.* Communication timeouts, link flapping, connection resets, and performance degradation interrupt execution.
- *Software Stack Failures.* Python exceptions, illegal memory accesses, OOM kills, and framework or library maintenance terminate workers.
- *Destructive Failures.* GPU loss, uncorrectable ECC errors, and driver or kernel faults reset or disable the device.

Figure 1 summarizes the distribution across the three traces; the characteristics of each cluster are as follows.

ByteDance. Software stack failures account for 70.8% of the reported interruptions, followed by user-driven interruptions at 17.3%, runtime anomalies at 10.7%, and destructive failures at only 1.2%.

Alibaba. Software stack failures account for 54.1% of interruptions, runtime anomalies for 11.7%, and destructive failures for 34.2%. Transient network errors, including NCCL timeouts, link flapping, and connection resets, are the largest individual source.

Shanghai AI Lab. User-driven interruptions account for 71.4% of events. The remaining events comprise software stack failures (18.0%), runtime anomalies (7.4%), and destructive failures (3.2%).

Based on failure severity, these causes can be grouped into two categories:

- *Destructive failure*: the GPU hardware, driver, or host OS suffers an unrecoverable fault (e.g., device loss, uncorrectable ECC errors, or a machine reboot), requiring the job to migrate to a new node.
- *Shallow disruption*: the GPU device and its software stack remain intact while only the user-space worker process exits. User-driven, runtime, and software stack interruptions all fall into this category.

Production systems already distinguish the two in practice: health-monitoring frameworks [2, 11, 32] continuously track GPU, driver, network, and OS status, and use these signals to select the appropriate recovery action. Under this classification, shallow disruptions account for 98.8%, 65.8%, and 96.8% of the reported interruptions in the three traces, respectively.

2.2 Recovery Overhead

GPU runtime state is large. LLM jobs maintain a large volume of runtime state on GPUs. After an interruption, the system must restore this state before it can resume work. The goal of recovery is to return to the progress achieved prior to the interruption. The state that must be recovered differs across workloads:

In the *training* scenario, the core state consists of model weights and optimizer states. Both are updated at every training step, with each update depending on the result of the previous step. In contrast, gradients and activations are intermediate products of each iteration: they are computed at the beginning of a step and released at its end, requiring no cross-step retention. With mixed-precision training using the Adam optimizer, the per-parameter memory footprint (weights, gradients, and optimizer states combined) is typically 16–20 bytes, so the total checkpoint size grows proportionally with the model parameter count.

In the *inference* scenario, model weights must be loaded from external storage onto the GPU each time a process starts. Besides weights, the inference system maintains a *KV cache*: it stores the intermediate key-value tensors from the attention computation of previously processed tokens, enabling subsequent generation to proceed incrementally without recomputing the full context. Together, model weights and the KV cache dominate GPU memory usage.

Motivation: checkpoint and reload overhead. Regardless of whether an interruption is a destructive failure or a shallow disruption, existing systems follow the same recovery procedure: checkpoint → restart → reload.

Table 1. Time overhead of checkpoint save and recovery in training and inference phases.

Phase	Operation & Method	Model & Time Overhead (s)			
		Llama3.2-1B	**Qwen3-1.7B**	**Llama3.2-3B**	
Training	Save (DeepSpeed)	24.98	36.05	66.20	
	Save (CheckFreq)	9.83	12.77	24.68	
	Save (In-memory)	*5.31*	*5.64*	*9.07*	
	Recovery (DeepSpeed)	24.01	28.84	42.86	
	Recovery (CheckFreq)	24.73	27.12	42.42	
	Recovery (In-memory)	*15.93*	*19.04*	*25.19*	
Inference		**Qwen3-8B**	**Qwen3-14B**	**Qwen3-32B**	**Qwen3-235B**
	Recovery (Replay)	38.48	56.47	52.22	**153.07**
	Recovery (KV-memory)	34.20	38.20	49.02	143.78

In the training scenario, checkpoints periodically save model parameters, optimizer states, and training progress to CPU memory or persistent storage. After an interruption, a new process reloads the full state from the most recent checkpoint and resumes training. To reduce checkpointing overhead, many prior works have proposed optimizations: writing snapshots to CPU memory first and flushing asynchronously to reduce foreground pause time [16, 27, 33]; compressing state to reduce write volume [37]; and leveraging data-parallel replicas for redundant recovery across workers to avoid reading from remote storage [7].

In the inference scenario, recovery overhead is equally significant. When a serving process restarts, the system must reload model weights and rebuild the KV cache. Recent inference fault-tolerance methods [24, 28, 39] continuously stream KV cache to remote GPUs or host memory during runtime, and copy it back to GPU memory during recovery.

Table 1 quantifies these overheads. We measure checkpoint save and recovery times for training, as well as recovery latency for inference, on an 8×A800 node. Each workload uses a single GPU, except Qwen3-235B, which uses all eight GPUs. For inference, model weights reside in the CPU page cache. In training, even with the fastest in-memory asynchronous scheme, saving Llama3.2-3B still requires 9.07 s and recovery takes 25.19 s; the DeepSpeed baseline requires 66.20 s for saving alone. In the inference scenario, recovering Qwen3-235B via replay takes 153.07 s, and the KV-memory scheme still requires 143.78 s. These overheads grow rapidly with model scale, and even after extensive optimization, the cost of saving and reloading state remains substantial.

3 Introducing Anchor

Existing systems apply the same recovery method regardless of whether the GPU is actually damaged: checkpoint → restart → reload. We investigate the following question: if a shallow disruption occurs, can we handle it in a better way?

3.1 Opportunity: In-place Restart

The recovery procedures for shallow disruptions and destructive failures appear identical: checkpoint → restart → reload. There is a key difference between the two: which GPU the new process lands on. Destructive failures require node replacement, whereas under shallow disruptions the OS kernel and hardware remain intact. Prior work [11, 14, 17] has exploited this property to restart jobs in-place. In-place restart avoids the overhead of node reallocation and environment re-initialization.

A pathological characteristic exists in the recovery process of shallow disruptions: the checkpoint mechanism first saves data out of the GPU and then loads it back onto the same GPU. We still cannot avoid the need for checkpointing: once a process exits, the CUDA driver reclaims all of the process's GPU memory. The data in GPU memory must therefore have a copy saved outside the GPU (i.e., a checkpoint). The root cause lies in the *fate-sharing* property of the GPU execution model: a CUDA context *owns* all GPU memory objects it allocates; when the process crashes, the context is destroyed and all memory is reclaimed. This calls for a mechanism that allows GPU memory objects allocated by a context to remain on the GPU even after that context exits.

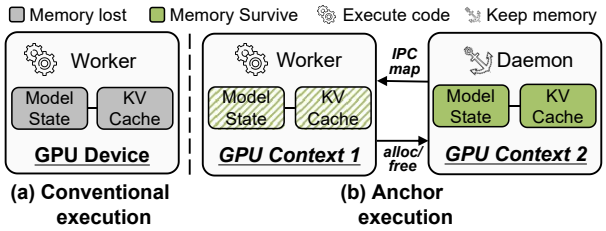


Figure 2. Comparison of conventional execution and our design. (a) In the conventional architecture, the worker owns all memory objects; a process exit loses everything. (b) Anchor lets the daemon own the memory object while the worker retains local execution; memory objects survive after the worker exits.

3.2 Insight: Decouple Memory Ownership

Our key insight is: *we should decouple memory ownership from the worker process.* We leverage the GPU IPC memory mechanism[1] provided by modern GPUs to achieve this decoupling. It possesses two key properties:

Reference counting: when a memory region is shared across multiple contexts via IPC, the GPU driver maintains a reference count for it. The exit of any sharing process only decrements the reference count and does not trigger actual reclamation of the memory. As long as at least one context still holds a reference, the memory remains alive.

Zero-overhead access via VM-PM mapping: the underlying implementation of IPC sharing inserts the owner's physical memory address into the consumer's GPU page table. Once the mapping is established, the consumer's accesses to that memory are served directly by the GPU hardware MMU/TLB, with no additional runtime overhead.

With the IPC mechanism, we can transfer GPU memory ownership from the worker to another process. As long as that process retains ownership of the GPU memory used by the worker, a shallow disruption cannot destroy it.

3.3 Anchoring a Single GPU Tensor

We describe how to use the IPC mechanism to anchor (i.e., retain the object through shallow disruptions) a single memory object on the GPU. As shown in Figure 2(b), we introduce a daemon process on the GPU to decouple the ownership

[1] `cudaIpcGet/OpenMemHandle` in CUDA; `hipIpcGet/OpenMemHandle` on AMD GPUs.

of the worker process's memory. Both the daemon process and the worker process have their own GPU contexts. When the worker needs to allocate a tensor, instead of allocating it itself, the daemon performs the allocation (and thus holds the ownership) on behalf of the worker. The daemon then grants access to the worker through an IPC mapping. Once the IPC mapping is established, the tensor appears identical to ordinary GPU memory in the worker's address space. The worker's computations on this tensor execute seamlessly within its own context. When the worker exits due to a shallow disruption, the memory held by the daemon is unaffected, owing to the reference-counting nature of the IPC mechanism. After a new worker starts, the daemon remaps the same GPU memory to it, and the new worker can access the tensor again. If implemented properly, we can anchor all GPU memory on the GPU, tens of gigabytes per GPU and hundreds of gigabytes per node, enabling efficient recovery from shallow disruptions.

3.4 Remaining Challenges

The mechanism above shows how to anchor a single GPU memory object. We extend this idea to a real ML system in §4, addressing two challenges.

Identity resolution. What the daemon retains are low-level GPU memory objects, whereas the upper-level framework operates on logical tensors created dynamically at runtime. When a new worker restarts, it must re-establish a one-to-one mapping between logical tensors and the correct memory objects retained by the daemon. A naive matching scheme based on allocation order is fragile, because users may modify the program between restarts, causing the allocation order to shift. The system therefore requires a stable identity resolution scheme that yields the same identity across restarts while remaining compatible with dynamic tensor creation and reclamation.

Consistency. Although the daemon holds ownership of the memory objects, the worker can freely read and write these objects during normal execution. If the worker crashes in the middle of an update, what the daemon retains may be a partially modified intermediate state. Merely retaining memory objects is not enough: the system must also guarantee that the state exposed after recovery corresponds to a semantically consistent recovery point.

4 Design

This section presents the design of ANCHOR. We first give an overview of the system architecture and workflow (§4.1), and then describe two key mechanisms: the rebinder (§4.2) and the consistency guard (§4.3).

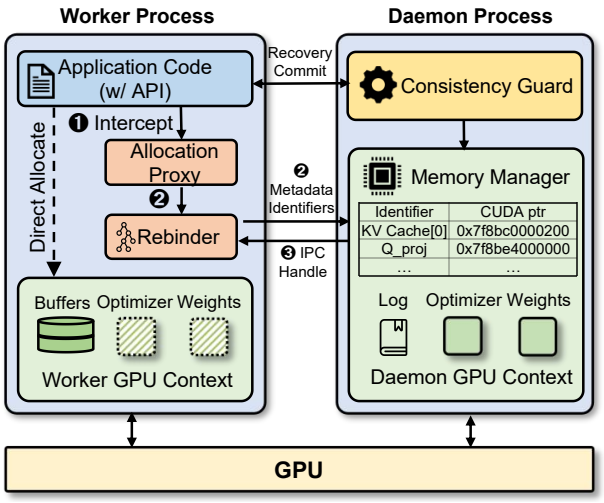


Figure 3. ANCHOR overview.

4.1 Overview and Workflow

Figure 3 shows the overview of ANCHOR, which consists of four components:

- **Allocation Proxy (§4.1).** Resides on the allocation path of the worker, intercepts allocation and deallocation requests within managed scopes, and translates them into protocol operations with the daemon.
- **Memory Manager (§4.1).** Runs inside the daemon, allocates GPU memory objects, and maintains the mapping from logical names to physical objects.
- **Rebinder (§4.2).** Generates stable identifiers for objects and, after a worker restart, rebinds logical tensors in the framework to GPU memory objects held by the daemon.
- **Consistency Guard (§4.3).** Ensures consistency of retained state. For training, it guarantees that the system recovers to the boundary of a completed optimizer step; for inference, it exposes only committed KV cache metadata and replays uncommitted suffixes.

Scope-based allocation. ANCHOR manages only those memory objects that need to be retained across restarts, such as model weights, optimizer states, and KV cache pages; short-lived objects like activations and temporary buffers still go through the native GPU allocation path. In this way, no extra interception is needed on the execution path. ANCHOR provides an API that transfers the ownership of objects allocated within user-marked scopes to the daemon. These scopes cover state creation and loading paths, such as model initialization, optimizer state construction, and KV cache setup. This design allows users to conveniently declare a batch of memory objects that need to be managed.

Figure 4 shows a typical usage example. The user only needs to wrap the model state initialization code with a `managed_scope`; the subsequent training loop requires no

```
# === Initialization: intercept allocations ===
with Anchor.managed_scope():
    model = LLaMA(config) # intercepted
    optimizer = Adam(model.params) # intercepted

# scope exit: naming + rebind + IPC mapping

# === Training: normal execution ===
for batch in dataloader:
    loss = model(batch)
    loss.backward()
    optimizer.step() # consistency protocol
```

Figure 4. User interface example of Anchor. The user only needs to wrap state initialization inside `managed_scope`; the training loop itself requires no modifications.

```
class LLMEngine:
    def __init__(self, config):
        # Standard nn.Module
        self.model = LLaMA(config)

        # KV cache is a native dict of raw tensors
        # Invisible to PyTorch's state_dict()
        # Access-path: engine.kv_cache['layer_0']
        self.kv_cache = {}
        for i in range(config.num_layers):
            # Intercepted in the managed_scope
            self.kv_cache[f"layer_{i}"] = torch.zeros(
                config.kv_shape, device="cuda"
            )

with Anchor.managed_scope():
    engine = LLMEngine(config)
```

Figure 5. Example of rebinding in a vLLM engine.

modifications. We modify the optimizer implementation to support the consistency protocol.

Workflow. The management of objects within a managed scope proceeds in three phases. ❶ Interception and placeholder creation. When execution enters a *managed allocation scope*, the Allocation Proxy begins intercepting memory allocation requests within it. For each allocation request, the worker first creates a lightweight placeholder tensor that holds the object's metadata and records how the object was created. The purpose is to satisfy the framework's need for the object's metadata while deferring global naming and object resolution until the scope exits. ❷ Identity resolution. When the scope exits, the rebinder scans all placeholder tensors, resolves an access path as the identifier for each object, and sends the identifiers along with the corresponding placeholder tensor metadata to the daemon. For each logical name, the daemon matches it against GPU memory objects that already exist from a previous run, or allocates a new object; it then returns the corresponding object's IPC handle to the worker. ❸ IPC mapping. The worker maps these IPC handles in its own GPU context and replaces the data pointer of each placeholder tensor with the IPC-mapped memory address. After this step, the worker can use these tensors for execution within its own context.

Deallocation and reclamation. When a managed object is deallocated in the program, the Allocation Proxy forwards the deallocation event to the daemon, which reclaims the corresponding GPU memory object. A special case is the normal exit of the worker process: Anchor detects the state of the Python interpreter and stops forwarding deallocation commands, so that the retained objects remain in the daemon and can be rebound and reused after restart.

4.2 Rebinder

When a worker restarts after a shallow disruption, it needs to re-establish the correspondence between logical tensors created by the upper-level framework at runtime and the physical GPU memory objects retained by the daemon. We call this the *rebinding problem.*

Limitations of existing naming methods. To achieve rebinding, we need to provide each memory object with an identifier that remains the same across multiple executions for matching purposes. Several straightforward identifier construction methods are infeasible: (1) Allocation sequence number: this method does not allow adding new tensors or removing existing ones. (2) Source code location: code refactoring causes line number shifts, and the same line within a loop body produces multiple distinct tensor instances. (3) Framework utilities: PyTorch `state_dict` utilities cover only `nn.Parameter` objects registered in the `nn.Module` hierarchy. As shown in Figure 5, tensors such as the KV cache exist in user applications that do not belong to any PyTorch module.

Naming by access path. We use the Python access path as the identifier for each object. In Python's execution model, all live objects form a directed object graph through attribute references and container indexing. As shown in Figure 5, Anchor obtains the variables of active frames on the call stack via `inspect.current_frame()` as traversal roots. Starting from these roots, every object can be located through a path composed of attribute accesses (`.attr`) and container indices (`[i]` / `[key]`), for example `engine.kv_cache['layer_0']`. We call this path the *access path* of the object. Because the access path is composed of indexing operations with clear application-level semantics, it is stable across multiple runs.

Handling aliases. Due to aliasing, the same object may have multiple access paths. For example, the embedding weights of a large language model are sometimes shared between the input and output layers. To obtain a unique

identifier, we adopt a deterministic search order: starting from the root set, we perform a breadth-first search (BFS), expanding nodes at the same level in lexicographic order of their accessors, and maintaining a visited set to break cycles that may exist in the object graph. The path at which each object is first reached during the BFS is the *access path* used by the engine; ANCHOR uses it as the stable identifier for each memory object.

Deferred naming. Computing access paths requires a complete BFS traversal of the object graph *top-down* from the root set. This traversal cannot be performed bottom-up because each object does not have information about its parent. Therefore, performing a full top-down traversal after every allocation would incur significant time overhead. ANCHOR defers the naming operation to the point when the managed scope exits. Inside the scope, each intercepted allocation request creates a *placeholder tensor*. When the scope exits, ANCHOR performs a single BFS traversal, collects all placeholder tensors that are still alive, and assigns canonical paths to them collectively. This design allows temporary objects that are created within the scope but released before exit to be excluded from naming.

Placeholder tensor. During the deferred naming period, the upper-level framework may need to query the metadata of allocated tensors, for example using shapes to determine the sizes of other buffers. To ensure transparency, ANCHOR creates *placeholder tensors*: a GPU memory object containing only a single physical element, whose metadata (shape, dtype, device) is set to be identical to the requested tensor. Its stride is set to all zeros, so that all logical indices map to the same physical element. This ensures that any metadata query (`.shape`, `.dtype`, `.stride()`) returns correct metadata while consuming minimal GPU memory. Since the lifetime of a placeholder is limited to the initialization code phase within the scope, during which the framework only reads metadata and does not execute actual computation kernels on the placeholder, the zero stride does not cause data correctness issues.

Placeholder tensors also record the initialization method of the allocation request. This information is used during first-time creation (when no prior state exists in the daemon) to enable the daemon to initialize the object in the same way. Common creation methods fall into three categories:

- *Shape-based* (e.g., `torch.empty(shape, dtype)`): ANCHOR directly records the allocation operator and its arguments.
- *Like-style* (e.g., `torch.zeros_like(src)`): ANCHOR extracts the source tensor's metadata and reduces it to a shape-based form.
- *Copy-style* (e.g., `tensor.clone()`, `tensor.to(device)`): the new object's data content depends on an existing source tensor. Since naming is deferred, the source tensor's GPU data may change before the scope exits; ANCHOR therefore buffers the source data to CPU at interception time to ensure correctness of deferred reconstruction.

Daemon matching. For each managed memory object identified by ANCHOR, the worker sends its access-path identifier and metadata to the daemon for matching. There are three possible outcomes: ❶ *Match*: the identifier already exists in the daemon and the metadata is compatible (dtype, shape, etc. match); the daemon returns the IPC handle of the existing memory object. ❷ *First-time creation*: the identifier does not exist in the daemon; the daemon performs a new allocation and returns the new IPC handle. ❸ *Conflict*: the identifier exists but the metadata is incompatible (e.g., shape or dtype differs); depending on the required policy, the daemon either reports the conflict and exits safely, or releases the old object, allocates a new one with the requested metadata, and returns the new handle. After obtaining the IPC handles, the worker replaces the data pointer of each placeholder tensor in place with the IPC-mapped memory address. This allows the application framework to use them seamlessly as ordinary tensors.

4.3 Consistency Guard

ANCHOR provides two kinds of consistency guarantees for existing GPU applications. We first describe how to provide a general-purpose consistency guarantee and its trade-offs, and then present lightweight consistency protocols designed for existing training and inference frameworks.

4.3.1 General Consistency. To guarantee application-agnostic consistency, we follow an approach similar to prior work PhoenixOS [34]. The mechanism consists of three parts: (1) Memory tracking: when the worker is about to launch a GPU kernel, the system first identifies which daemon-managed memory regions the kernel will modify. We track the memory set that the application is about to write by intercepting and analyzing the API call parameters of the kernel, and validate the working set at runtime. (2) GPU-resident copy-on-write: after identifying the set of memory regions to be written, ANCHOR triggers a CoW operation before the kernel actually executes. The system allocates a GPU buffer in the daemon and backs up the data that is about to be modified into that buffer. (3) Group-based rollback and commit: we treat the modifications of a group of kernels as a single atomic unit, such as one step in training or inference (executing all layers). This is because inter-layer activations are not saved, making partial-layer rollback infeasible. If all kernels within the group complete execution, we release the CoW temporary buffer; if an interruption occurs, we roll back to the previous version.

Trade-offs. This mechanism is transparent to user applications and achieves consistency without any code modifications. However, this *black-box* consistency guarantee

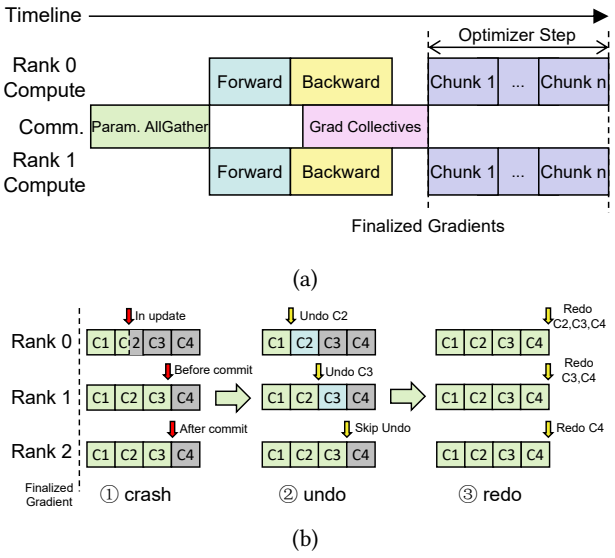


Figure 6. Training consistency in ANCHOR. (a) Training pipeline of one iteration. (b) Recovery of three ranks interrupted at different chunk positions.

introduces trade-offs in modern ML systems. First, the tracking mechanism requires intercepting and analyzing API and kernel behavior, incurring runtime overhead. More importantly, the CoW mechanism introduces substantial memory overhead. For example, when updating optimizer states or model weights, the CoW mechanism backs up the full data in GPU memory, which can amount to tens of gigabytes.

To reduce the overhead of ANCHOR under ML application frameworks, we introduce a second kind of consistency guarantee: lightweight consistency protocols based on application semantics.

4.3.2 Log-based Training Consistency.

In training, consistency is defined as: the model state (weights and optimizer states) after completing a training iteration is consistent with the content of the corresponding checkpoint version. ANCHOR uses a low-overhead log-based recovery protocol.

Distributed training iteration. Figure 6(a) shows the training pipeline of one iteration. During the backward pass, gradient collectives overlap with computation. Existing frameworks (e.g., DeepSpeed, FSDP) require all collectives to complete before the optimizer step begins; we call this synchronization point the *gradient-finalization boundary*.

The optimizer then updates the shard in a fixed sequence of *chunks*. A chunk groups the parameters and optimizer states that a single optimizer invocation writes together. A disruption before the boundary leaves the model state at the previous consistent version, so recovery simply recomputes the iteration. A disruption after the boundary, however, may leave a mixture of updated and stale chunks, requiring the protocol below.

Algorithm 1 Training Consistency Method (Per-Rank)

▷ *chunk*[ ], *grad*[ ], *undo_buf*, *cursor*, *and undo_chunk are retained by the daemon and survive disruptions.*

procedure ApplyChunk(i)
  undo_buf ← Copy(*chunk*[i])
  undo_chunk ← i ▷ publish complete undo
  Optimizer.Apply(*chunk*[i], *grad*[i])
  cursor ← $i+1$ ▷ commit chunk
end procedure

▷ *Normal Path*

cursor ← 0; *undo_chunk* ← ⊥
for $i = 0$ **to** $N-1$ **do**
  ApplyChunk(i)
end for

▷ *Recovery Path*

if *cursor* < N **then** ▷ disruption during step
  if *undo_chunk* = *cursor* **then**
    chunk[*cursor*] ← Copy(*undo_buf*) ▷ rollback
  end if
  for j = *cursor* **to** $N-1$ **do**
    ApplyChunk(j) ▷ redo
  end for
end if

Observation: finalized gradients as a free redo log. The gradient-finalization boundary ensures that gradients are immutable throughout the optimizer step. Thus the current model state, finalized gradients, and per-step metadata fully determine the next version. By retaining the gradients and metadata until the step completes, ANCHOR can replay any unfinished chunk updates using the existing gradient buffers as a **redo log** with no extra memory overhead. One gap remains: the chunk under modification may be partially written, so directly reapplying it could update some elements twice. ANCHOR closes this gap with a single *undo buffer* for the active chunk; the full protocol is given in Algorithm 1.

Consistency method. ANCHOR applies the undo/redo protocol of Algorithm 1 at chunk granularity. After the gradient-finalization boundary, each rank independently updates the chunks of its own parameter shard without cross-rank coordination. ANCHOR assigns each chunk a fixed position index and maintains a *cursor* recording the first uncommitted chunk, and a single *undo buffer* holding a pre-update copy of the chunk at the cursor.

During normal execution, before updating chunk i, ANCHOR copies it into the undo buffer; after the update completes, it advances the cursor to i+1. During recovery, if the undo buffer covers the cursor chunk, ANCHOR restores that chunk first, then replays from the cursor to the end using the redo log.

Recovery illustration. Figure 6(b) shows three ranks interrupted at different positions after the gradient-finalization boundary. Rank 0 is interrupted while modifying C2: it restores C2 from the undo buffer and reapplies C2–C4. Rank 1 has completed C3's update but not advanced the cursor: it restores C3 and reapplies C3–C4. Rank 2 has already committed C3: it skips rollback and reapplies only C4. In all cases, each rank recovers to the state in which all chunks have been updated.

Overhead analysis. The general consistency protocol duplicates the full model weights and optimizer states via CoW. In contrast, ANCHOR requires only an undo buffer the size of one recovery chunk, and reuses the existing gradient buffers as the redo log with no additional allocation. In DeepSpeed ZeRO-3 [25], a recovery chunk holds the local partition of one parameter sub-group, bounded by `sub_group_size` (default: 10^9 elements).

4.3.3 Inference Consistency.

In inference, consistency is defined as: after an interruption, the KV cache is in the state of a completed inference step, allowing subsequent generation to proceed correctly.

Observation: KV-cache entries are append-only. Under PagedAttention [13], KV-cache entries are *append-only*. The inference framework directly appends the KV cache of the currently processed new token(s) (a chunk during prefill or a single token during decode) into the corresponding slot(s) of the KV cache page, and then invokes the attention kernel for computation. In the worst case, the worker crashes in the middle of kernel execution, leaving the KV cache slot(s) polluted by a partial write. During recovery, the inference engine re-executes the forward pass for the in-flight token(s) and overwrites the same KV cache slot(s). The dirty data is overwritten with correct data.

Consistency protocol. Based on the blind-write property, ANCHOR guarantees consistency through CPU-side metadata management:

- **Normal execution:** The worker freely writes new KV data into the currently allocated slots in GPU memory. After a decode step or chunked prefill completes, ANCHOR commits the metadata for that sequence on the CPU side (block table mappings, sequence length cursor).
- **Interruption recovery:** If the worker crashes before the metadata commit, the system loads the last committed metadata from the daemon during recovery to reconstruct the page table view. The engine resumes execution from this point; the KV cache slots involved in the interrupted forward pass are overwritten upon re-execution, requiring no explicit cleanup.

Overhead analysis. This approach incurs no GPU memory overhead, persisting only a small amount of page table metadata on the CPU side. During recovery, it only needs to discard the suffix of the last uncommitted operation and re-execute the forward pass. For each request, the cost is recomputing a single (decode) token or one prefill chunk.

4.4 Discussion

Rebinding assumptions. Rebinding relies on access paths to re-establish the correspondence between objects. However, due to the complexity of existing frameworks, even when the access path is the same, the semantics may change. For example, DeepSpeed partitions the optimizer states of multiple parameters into groups of a fixed size. If a new weight is added to the model structure, the tensors contained in the affected group and all subsequent groups will change. We cannot support modifications where the tensor name remains the same but its semantics change substantially.

Failure coverage and checkpoint fallback. The daemon has a much smaller failure surface than a worker because it only manages memory allocation and IPC, without executing user-defined CPU or GPU logic. ANCHOR can rapidly recover from a shallow disruption as long as the daemon remains alive. To retain coverage for failures outside this scope, applications can continue using their existing checkpointing mechanisms alongside ANCHOR. ANCHOR conservatively falls back to the latest application checkpoint in three cases: (1) the failure is destructive; (2) the daemon is dead; or (3) the failure cannot be confidently classified as shallow.

Silent errors. The preceding failure model covers explicit disruptions that surface through worker exits or infrastructure health signals. SDC and incorrect writes from buggy kernels can silently corrupt GPU state without such signals; they are difficult for production systems to detect, and ANCHOR does not target them either. These faults are outside the scope of ANCHOR and must be addressed using specialized GPU diagnosis [41] together with application-level validation [10]; when corruption is suspected, the system rolls back to a known-good checkpoint.

Scope of the lightweight protocols. The lightweight consistency protocols rely on several framework properties. The training protocol assumes a gradient finalization boundary, the inference protocol assumes that KV cache updates only append, and current training frameworks [25, 40] and inference engines [13] satisfy both. Frameworks and models that fall outside these assumptions, such as optimizers that fuse parameter updates into the backward pass or decoders with linear attention that maintain a recurrent state modified in place [12], can use the general protocol (§4.3.1).

5 Implementation

ANCHOR is implemented entirely in user space, based on PyTorch 2.9.1, DeepSpeed 0.17.7 [25], and vLLM 0.14.1 [13]. The

intrusion of ANCHOR into existing ML frameworks is minimal. The modifications to each ML application framework amount to 200–300 lines of code.

5.1 Allocation Interception

PyTorch enables the caching allocator by default. To reduce allocation overhead, the caching allocator performs large-granularity physical memory allocations and then allocates logical memory blocks within the framework. This prevents the daemon from identifying the specific memory objects under management. ANCHOR disables PyTorch's caching allocator within the managed scope. Since the daemon does not frequently allocate and free memory during initialization, disabling it does not cause memory fragmentation and does not incur significant overhead. It also ensures that these memory regions are exclusively managed by the daemon.

5.2 Multi-GPU Support

ANCHOR easily extends to multi-GPU settings. On each GPU, ANCHOR launches an independent daemon for that device. Each worker process communicates exclusively with the daemon on its local GPU. This per-device daemon design requires no cross-device coordination at the ANCHOR level. For inter-GPU communication, libraries such as NCCL allocate their own internal buffers (e.g., communication rings and scratch memory) inside the worker process. These buffers are placed outside the managed allocation scope and remain local to the worker.

6 Evaluation

6.1 Experimental Setup

Platform. Our primary evaluation uses an 8-GPU NVIDIA server running Ubuntu 22.04.5 LTS with Linux kernel 5.15.0. The machine is equipped with two Intel Xeon Platinum 8358 CPUs (64 physical cores / 128 hardware threads in total), 1 TiB of system memory, and eight NVIDIA A800-SXM4-80GB GPUs. The GPUs are fully connected by NVLink within the node, and the machine exposes two NUMA domains aligned with the 8-GPU topology. For persistent storage, our experiments use a 256 GB NVMe-backed shared filesystem; in-memory recovery experiments additionally use a 400 GB tmpfs-backed shared-memory region. All experiments in Sections 6.2–6.5 use this platform; the portability study instead uses a single-GPU AMD server with an AMD Instinct MI100 running ROCm 6.4.3-128.

Workloads. We evaluate training, where ANCHOR preserves model parameters and optimizer state, and inference, where it preserves model weights and KV caches. Training uses the WikiText-2 raw-v1 corpus with three single-GPU models—Llama3.2-1B, Qwen3-1.7B, and Llama3.2-3B—and one 8-GPU model, Qwen2.5-14B. All training runs use BF16 with DeepSpeed ZeRO-3; checkpoint-based baselines save

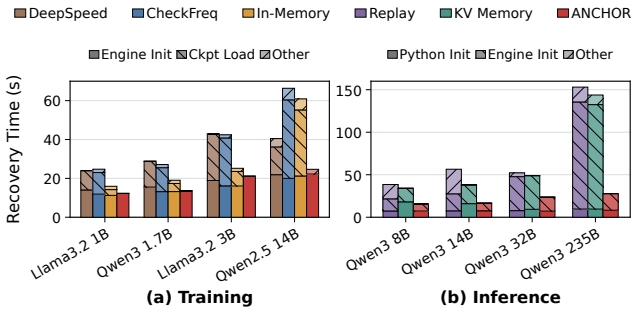


Figure 7. Fault-recovery breakdowns for training and inference.

every 100 steps, and end-to-end runs execute 2,000 optimization steps. Inference uses cleaned ShareGPT traces at fixed concurrency with three single-GPU models—Qwen3-8B, Qwen3-14B, and Qwen3-32B—and one 8-GPU model, Qwen3-235B. Each inference run processes the same 10,000-request workload, preserving the arrival mix and output-length distribution across recovery schemes.

Failure model. We focus on non-destructive failures that terminate the worker process without resetting the GPU device. Under this failure model, GPU memory contents remain intact, which matches the target scenario of ANCHOR. For both training and inference, we inject intermittent worker crashes during execution, restart the worker in place, and measure the time required to reconnect to the preserved GPU state and resume useful work. The per-second fault probability is 0.1% for training and 1% for inference. This setup provides a controlled and repeatable way to evaluate recovery from shallow disruptions.

Baselines and metrics. For training, we compare against *DeepSpeed native*, which checkpoints to disk; *CheckFreq*, which stages snapshots in CPU memory before asynchronous disk writes; and *In-Memory checkpoint*, which stores snapshots in memory. For inference, *Replay* rebuilds active requests and recomputes prefill, while *KV-Memory* restores a CPU checkpoint of the KV cache. In contrast, ANCHOR retains model, optimizer, and KV state in GPU memory and reattaches it through IPC after restart. We report recovery latency and its startup/restoration breakdown, end-to-end performance under failures, and failure-free overhead. The corresponding metrics are training throughput and step time, inference latency and token throughput, and steady-state GPU memory usage.

6.2 Save and Recovery Latency

Figure 7 breaks recovery into startup and state restoration. Table 2 reports normal-path persistence latency. The operations differ: baselines create full snapshots, while ANCHOR protects one recovery chunk already resident on the GPU.

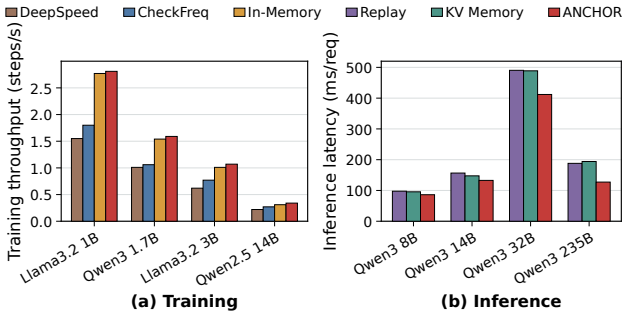


Figure 8. End-to-end performance under injected failures.

Training. ANCHOR cuts average recovery latency by 26.6% against the strongest baseline. The baselines initialize the framework and rebuild model and optimizer state from the latest snapshot. ANCHOR still pays for framework startup. It reattaches surviving GPU objects instead of rebuilding them. Load-or-attach time therefore nearly disappears, leaving startup as the dominant component. The undo and redo records recover the latest optimizer state, avoiding rollback to an earlier checkpoint.

On the normal path, copying one optimizer subgroup into the undo buffer takes 0.03–0.07 s. The checkpoint operations in Table 2 take 5.31–66.76 s because they materialize full-state snapshots, but this difference is not a direct speedup: the operations cover different amounts of state and execute at different frequencies. The relevant result is that chunk-level protection is sufficiently small to produce the low steady-state overhead reported in Section 6.5.

Inference. ANCHOR cuts average recovery latency by 60.7% against the strongest baseline. On Qwen3-235B, KV-memory needs 143.78 s; ANCHOR needs 27.85 s, a 5.16× speedup. Replay rebuilds active requests and recomputes their prefills. KV-memory copies checkpointed KV state back from CPU memory. Both also rebuild the inference engine around restored state. By preserving both weights and KV cache, ANCHOR removes these state-restoration paths. As in training, engine startup then dominates the remaining latency. The larger inference gain comes from eliminating the substantial model and request state that the baselines reconstruct after each restart.

Method	Llama3.2-1B	Qwen3-1.7B	Llama3.2-3B	Qwen2.5-14B
DeepSpeed native	24.98	36.05	66.20	66.76
CheckFreq	9.83	12.77	24.68	35.61
In-memory checkpoint	5.31	5.64	9.07	28.16
ANCHOR	0.03	0.04	0.07	0.05

Table 2. Blocking latency of training-side persistence operations. Baselines create full-state checkpoints; ANCHOR copies one recovery chunk.

6.3 End-to-End Performance Under Failures

Figure 8 reports complete runs with injected failures. It includes normal-path overhead, recovery downtime, and work lost after each failure.

Training. Across models, ANCHOR improves end-to-end throughput by 5.1% over the strongest baseline. Its gain reaches 1.81× over DeepSpeed native. Checkpoint schemes pay both periodic persistence cost and post-failure checkpoint loading, then recompute the steps completed after the last checkpoint. ANCHOR instead preserves completed progress in GPU memory. A failure before gradient finalization requires only the interrupted iteration to be recomputed; a failure during the optimizer step rolls back the current recovery chunk and completes the update from retained gradients. This avoids checkpoint-interval rollback and leaves a larger fraction of the 2,000-step run for useful optimization. The end-to-end gain is smaller than the isolated recovery improvement because failure-free computation still dominates total execution time.

Inference. Across models, ANCHOR reduces end-to-end latency by 17.0% relative to the strongest baseline. For each restart, preserved weights and KV state avoid model reload, CPU-to-GPU KV restoration, and prefill recomputation. These savings accumulate across the 10,000-request run, while request execution between failures is unchanged. The 17.0% end-to-end reduction is therefore lower than the 60.7% recovery reduction, as expected, but shows that the isolated recovery benefit remains visible at the service level.

6.4 Correctness

We validate the correctness of the design points introduced by ANCHOR through controlled fault-injection tests of rebinding and the training consistency protocol.

Rebinding. We evaluate the eight program changes summarized in Table 3, covering tensor addition, removal, renaming, reordering, metadata changes, and semantic changes. To expose the matching behavior, we configure the daemon to report a metadata conflict and exit safely. For each case, a fresh execution of the modified program without rebinding serves as the ground truth. We mark a result as correct when its inference output matches the ground truth.

Case	Program change	Alloc. order	ANCHOR
(a)	Enable a LoRA adapter	✓	✓
(b)	Remove visual encoder	▲	✓
(c)	Enable INT8 quantization	▲	✓
(d)	Rename KV-cache tensors	✓	✓
(e)	Swap attention/MLP initialization order	▲	✓
(f)	Remove final decoder layer	▲	✓
(g)	Reduce KV cache capacity	▲	▲
(h)	Change the KV cache layout	✗	✗

✓ Correct ▲ Safe rejection ✗ Incorrect

Table 3. Rebinding outcomes across eight program changes.

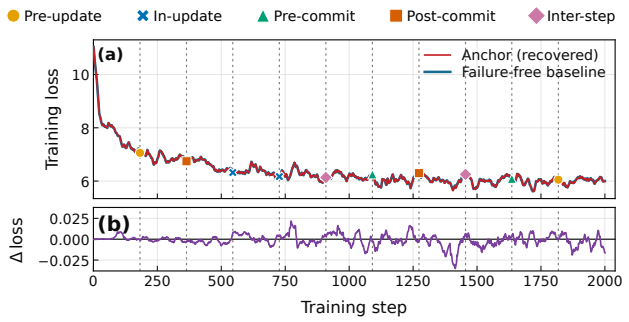


Figure 9. Training-state consistency under injected failures: (a) recovered and failure-free training-loss trajectories; (b) their difference (recovered minus failure-free).

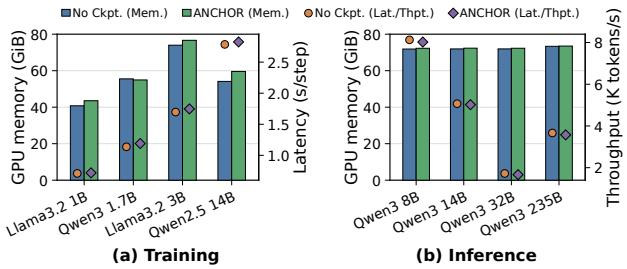


Figure 10. Overhead of Anchor.

Table 3 shows that Anchor correctly handles six cases, whereas allocation-order matching handles only two. In cases (b), (c), (e), and (f), the allocation order changes while the identifiers of the daemon-managed objects remain unchanged. Specifically, enabling INT8 quantization in case (c) introduces additional scale tensors in each layer and shifts subsequent allocations. Anchor successfully matches these objects, whereas allocation-order matching becomes misaligned. In case (a), the LoRA state is added after the base-model state without changing the preceding allocation order, allowing both methods to reuse the base-model state and allocate the new LoRA state.

Cases (d) and (g) represent changes visible in the identifier name or metadata. In case (d), renaming a tensor causes Anchor to perform recreation rather than match an existing object. In case (g), incompatible metadata causes the daemon to report a conflict and exit safely..

In case (h), the operator switches the KV cache layout (e.g., NHD to HND) by changing the attention backend. This changes how the cached data is interpreted without altering the tensor's identifier, shape, or dtype, so both methods silently reuse the incompatible data. This illustrates a fundamental limit: Anchor cannot detect a semantic change that leaves identifiers and metadata unchanged.

Training-state consistency. To validate the training-state consistency of Anchor, we repeatedly inject worker failures at five training stages: pre-update, in-update, pre-commit, post-commit, and inter-step (Figure 9). These injection points cover failures before, during, and after an optimizer update, including both sides of the commit boundary and the interval between two steps. We compare the training-loss trajectory of the recovered 2,000-step run with that of a failure-free run, together with their per-step difference.

Figure 9(a) shows that the recovered and failure-free loss trajectories closely overlap. Figure 9(b) further shows that the magnitude of their smoothed per-step difference remains below 2.5×10^{-2}. Small differences are expected because the same GPU kernels may produce slightly different floating-point results across runs. These results indicate that Anchor recovers consistently at the tested stages, successfully recovering from all 10 injected failures across the five stages.

6.5 Failure-Free Overhead

Figure 10 compares Anchor with the native failure-free baseline after each workload reaches steady state. Unlike the individual persistence operations in Table 2, this experiment measures the recurring cost under the same application workload and is the appropriate normal-path comparison.

Training. Across models, Anchor uses 4.9% more GPU memory and increases step time by 2.6% on average. The largest step-time overhead is 4.7%. Most extra memory holds the one-chunk undo buffer, not a full copy of model state. Existing finalized gradients serve as the redo log; Anchor retains them without another GPU copy. Step-time overhead comes from the pre-update chunk copy and cursor advancement. The training path does not serialize a full checkpoint every step.

Inference. Across models, Anchor uses 0.45% more GPU memory and reduces token throughput by 2.0% on average. The maximum throughput reduction is 3.2%. Anchor does not duplicate model weights or KV payloads. Its consistency protocol maintains per-sequence metadata and commits it at decode or chunked-prefill boundaries. The small costs show that retaining attachable state does not require copying the inference working set continuously. Together, the training and inference results show that direct reattachment lowers recovery cost while adding modest steady-state overhead.

6.6 Generality on AMD GPUs

We repeat the training persistence and recovery experiments on an AMD Instinct MI100 using ROCm/HIP IPC. Figure 11 shows 17.3–19.0 s recovery latency for Anchor, compared with 19.9–28.7 s for the baselines. The breakdown matches the NVIDIA result: reattachment removes most restoration work. Framework initialization remains the main cost. This indicates that the recovery mechanism does not depend on CUDA's particular IPC implementation.

Method	Llama3.2-1B	Qwen2.5-1.5B	Qwen3-0.6B
DeepSpeed native	16.81	21.32	10.02
CheckFreq	4.06	5.17	2.57
In-memory checkpoint	3.89	4.17	2.48
ANCHOR	1.07	1.76	0.04

Table 4. Normal-path persistence latency on AMD. Baselines save full state; ANCHOR copies one recovery chunk.

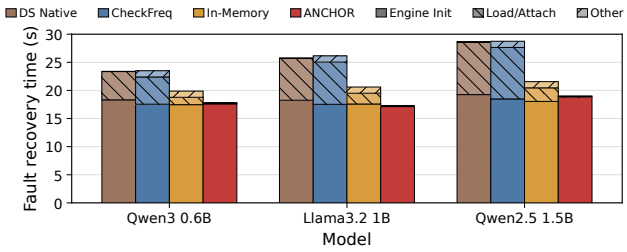


Figure 11. AMD training fault-recovery breakdown.

Table 4 reports 0.04–1.76 s for copying one recovery chunk and 2.48–21.32 s for creating the baselines' full-state checkpoints. As with the NVIDIA results, these values characterize different operations and should not be interpreted as a save-time speedup. Together with the recovery breakdown, they show that the chunk protocol remains lightweight under ROCm while preserved state avoids reconstruction.

7 Related Work

Checkpoint-based Recovery. Checkpoint-based recovery for GPU workloads can be broadly divided into periodic and just-in-time approaches. Periodic checkpointing saves model and runtime states to persistent storage or CPU memory at regular intervals, and recent systems [15, 16, 18, 22, 27, 31, 33, 38] reduce its overhead through asynchronous checkpointing, checkpoint-layout optimization, and in-memory placement. Just-in-time checkpointing [7, 35], in contrast, only needs to pay most of the recovery cost on the failure path to enable fast recovery with low steady-state overhead. At a lower level, process-transparent GPU checkpoint/restart systems such as PhOS [34] and NVCR [19] follow the same paradigm by capturing execution state and later restoring it for resumed execution. Despite these differences, all these approaches still follow a save-and-reconstruct model: they explicitly materialize recoverable state outside the running worker and later rebuild execution from that state after failure. By contrast, our design avoids checkpoint overhead by directly reusing surviving GPU-resident state when non-destructive disruptions happen.

Proxy-based GPU Systems. Singularity [26] intercepts device APIs and ships them to a device proxy, which makes GPU jobs transparently checkpointable and migratable. Because the proxy issues the kernels, a kernel fault can crash the proxy, and the GPU must then be rebuilt by replaying the calls that changed its state. NVIDIA Multi-Process Service (MPS) [21] raises utilization by multiplexing concurrent clients through a persistent server, but the clients on a GPU share a fault domain, and a fatal fault in any one of them forces every client on that GPU to exit. The daemon of ANCHOR has a narrow failure surface: it executes only memory allocation and IPC while kernels run in the worker, so GPU state survives a worker fault and a restarted worker can reattach to it.

Cross-process GPU Resource Access. Prior work has used IPC and related cross-process mechanisms in GPU systems. Common GPU communication libraries like NCCL [20], UCX [29], and UCC [30] optimize collective operations across different processes by enabling kernel direct access in HPC and deep learning workloads. ServerlessLLM [3] uses CUDA IPC handles to allow an auxiliary process to concurrently load the checkpoint when initializing the inference engine. Overall, these systems use IPC primarily to support concurrent communication, sharing, or access to GPU resources across live processes. In contrast, our work uses CUDA IPC to decouple memory-object lifetime from worker lifetime.

Virtual-Physical Memory Decoupling. GMLake [5] uses virtual memory stitching to defragment GPU memory for DNN training, while vAttention [23] and vTensor [36] use CUDA VMM for flexible KV-cache allocation and remapping. These systems decouple virtual and physical memory within a running process. In contrast, ANCHOR decouples memory ownership across process lifetimes, preserving GPU objects for reuse after worker disruptions.

8 Conclusion

We presented ANCHOR, a failure-aware recovery system for shallow disruptions in GPU workloads. By decoupling GPU-memory ownership from the worker process, ANCHOR preserves long-lived GPU state across crashes and allows a restarted worker to reattach it directly. To make this practical for real ML systems, we further designed a stable tensor-rebinding mechanism and lightweight consistency mechanisms for both training and inference. Experiments on DeepSpeed and vLLM show that ANCHOR substantially reduces recovery latency, improves end-to-end efficiency under failures, and incurs only modest steady-state overhead.

Acknowledgments

This work is supported by the National Key R&D Program of China (Grant No. 2024YFB4505201), the National Natural Science Foundation of China (Grant No. U22B2023, 62572296, 62332011), the Beijing Natural Science Foundation (Grant No. L242016), and the High Performance Computing Center of Tsinghua University.

References

[1] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, and Marcel Blistein et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. arXiv:2507.06261 [cs.CL] https://arxiv.org/abs/2507.06261

[2] Yangtao Deng, Xiang Shi, Zhuo Jiang, Xingjian Zhang, Lei Zhang, Zhang Zhang, Bo Li, Zuquan Song, Hang Zhu, Gaohong Liu, Fuliang Li, Shuguang Wang, Haibin Lin, Jianxi Ye, and Minlan Yu. 2025. Minder: Faulty Machine Detection for Large-scale Distributed Model Training. arXiv:2411.01791 [cs.DC] https://arxiv.org/abs/2411.01791

[3] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 8, 19 pages.

[4] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and Amy Yang et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[5] Cong Guo, Rui Zhang, Jiale Xu, Jingwen Leng, Zihan Liu, Ziyu Huang, Minyi Guo, Hao Wu, Shouren Zhao, Junping Zhao, and Ke Zhang. 2024. GMLake: Efficient and Transparent GPU Memory Defragmentation for Large-scale DNN Training with Virtual Memory Stitching. arXiv:2401.08156 [cs.DC] https://arxiv.org/abs/2401.08156

[6] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, and Shirong et al. Ma. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (Sept. 2025), 633–638. doi:10.1038/s41586-025-09422-z

[7] Tanmaey Gupta, Sanjeev Krishnan, Rituraj Kumar, Abhishek Vijeev, Bhargav Gulavani, Nipun Kwatra, Ramachandran Ramjee, and Muthian Sivathanu. 2024. Just-In-Time Checkpointing: Low Cost Error Recovery from Deep Learning Training Failures. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) *(EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 1110–1125. doi:10.1145/3627703.3650085

[8] Tao He, Xue Li, Zhibin Wang, Kun Qian, Jingbo Xu, Wenyuan Yu, and Jingren Zhou. 2023. Unicron: Economizing Self-Healing LLM Training at Scale. arXiv:2401.00134 [cs.DC] https://arxiv.org/abs/2401.00134

[9] Qinghao Hu, Zhisheng Ye, Zerui Wang, Guoteng Wang, Meng Zhang, Qiaoling Chen, Peng Sun, Dahua Lin, Xiaolin Wang, Yingwei Luo, Yonggang Wen, and Tianwei Zhang. 2024. Characterization of Large Language Model Development in the Datacenter. arXiv:2403.07648 [cs.DC] https://arxiv.org/abs/2403.07648

[10] Yuxuan Jiang, Ziming Zhou, Boyu Xu, Beijie Liu, Runhui Xu, and Peng Huang. 2025. Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 313–329. https://www.usenix.org/conference/osdi25/presentation/jiang

[11] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: scaling large language model training to more than 10,000 GPUs. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation* (Santa Clara, CA, USA) *(NSDI'24)*. USENIX Association, USA, Article 41, 16 pages.

[12] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Francois Fleuret. 2020. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. arXiv:2006.16236 [cs.LG] https://arxiv.org/abs/2006.16236

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[14] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. 2020. PyTorch distributed: experiences on accelerating data parallel training. *Proc. VLDB Endow.* 13, 12 (Aug. 2020), 3005–3018. doi:10.14778/3415478.3415530

[15] Avinash Maurya, Robert Underwood, M. Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2024. DataStates-LLM: Lazy Asynchronous Checkpointing for Large Language Models. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '24)*. ACM, 227–239. doi:10.1145/3625549.3658685

[16] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. 2021. CheckFreq: Frequent, Fine-Grained DNN Checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 203–216. https://www.usenix.org/conference/fast21/presentation/mohan

[17] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: a distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation* (Carlsbad, CA, USA) *(OSDI'18)*. USENIX Association, USA, 561–577.

[18] Bogdan Nicolae, Jiali Li, Justin M. Wozniak, George Bosilca, Matthieu Dorier, and Franck Cappello. 2020. DeepFreeze: Towards Scalable Asynchronous Checkpointing of Deep Learning Models. In *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. 172–181. doi:10.1109/CCGrid49817.2020.00-76

[19] Akira Nukada, Taichiro Suzuki, and Satoshi Matsuoka. 2023. Efficient checkpoint/Restart of CUDA applications. *Parallel Comput.* 116, C (July 2023), 9 pages. doi:10.1016/j.parco.2023.103018

[20] NVIDIA. 2025. NVIDIA Collective Communications Library (NCCL). https://developer.nvidia.com/nccl.

[21] NVIDIA. 2025. NVIDIA Multi-Process Service (MPS). https://docs.nvidia.com/deploy/mps/.

[22] James S. Plank, Kai Li, and Michael A. Puening. 1998. Diskless Checkpointing. *IEEE Trans. Parallel Distrib. Syst.* 9, 10 (Oct. 1998), 972–986. doi:10.1109/71.730527

[23] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. arXiv:2405.04437 [cs.LG] https://arxiv.org/abs/2405.04437

[24] Shangshu Qian, Kipling Liu, P. C. Sruthi, Lin Tan, and Yongle Zhang. 2026. Towards Resiliency in Large Language Model Serving with KevlarFlow. arXiv:2601.22438 [cs.DC] https://arxiv.org/abs/2601.22438

[25] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Virtual Event, CA, USA) *(KDD '20)*. Association for Computing Machinery, New York, NY, USA, 3505–3506. doi:10.1145/3394486.3406703

[26] Dharma Shukla, Muthian Sivathanu, Srinidhi Viswanatha, Bhargav Gulavani, Rimma Nehme, Amey Agrawal, Chen Chen, Nipun Kwatra,

Ramachandran Ramjee, Pankaj Sharma, Atul Katiyar, Vipul Modi, Vaibhav Sharma, Abhishek Singh, Shreshth Singhal, Kaustubh Welankar, Lu Xun, Ravi Anupindi, Karthik Elangovan, Hasibur Rahman, Zhou Lin, Rahul Seetharaman, Cheng Xu, Eddie Ailijiang, Suresh Krishnappa, and Mark Russinovich. 2022. Singularity: Planet-Scale, Preemptive and Elastic Scheduling of AI Workloads. arXiv:2202.07848 [cs.DC] https://arxiv.org/abs/2202.07848

[27] Foteini Strati, Michal Friedman, and Ana Klimovic. 2025. PCcheck: Persistent Concurrent Checkpointing for ML. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 811–827. doi:10.1145/3669940.3707255

[28] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. 2024. DéjàVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving. arXiv:2403.01876 [cs.DC] https://arxiv.org/abs/2403.01876

[29] Yiltan Hassan Temucin, Amir Hossein Sojoodi, Pedram Alizadeh, Benjamin Kitor, and Ahmad Afsahi. 2022. Accelerating Deep Learning Using Interconnect-Aware UCX Communication for MPI Collectives. *IEEE Micro* 42, 2 (March 2022), 68–76. doi:10.1109/MM.2022.3148670

[30] Manjunath Gorentla Venkata, Valentine Petrov, Sergey Lebedev, Devendar Bureddy, Ferrol Aderholdt, Joshua Ladd, Gil Bloch, Mike Dubman, and Gilad Shainer. 2025. Unified Collective Communication: A Unified Library for CPU, GPU, and DPU Collectives. *IEEE Micro* 45, 2 (March 2025), 26–35. doi:10.1109/MM.2025.3534638

[31] Borui Wan, Mingji Han, Yiyao Sheng, Yanghua Peng, Haibin Lin, Mofan Zhang, Zhichao Lai, Menghan Yu, Junda Zhang, Zuquan Song, Xin Liu, and Chuan Wu. 2025. ByteCheckpoint: a unified checkpointing system for large foundation model development. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation* (Philadelphia, PA, USA) *(NSDI '25)*. USENIX Association, USA, Article 30, 20 pages.

[32] Borui Wan, Gaohong Liu, Zuquan Song, Jun Wang, Yun Zhang, Guangming Sheng, Shuguang Wang, Houmin Wei, Chenyuan Wang, Weiqiang Lou, Xi Yang, Mofan Zhang, Kaihua Jiang, Cheng Ren, Xiaoyun Zhi, Menghan Yu, Zhe Nan, Zhuolin Zheng, Baoquan Zhong, Qinlong Wang, Huan Yu, Jinxin Chi, Wang Zhang, Yuhan Li, Zixian Du, Sida Zhao, Yongqiang Zhang, Jingzhe Tang, Zherui Liu, Chuan Wu, Yanghua Peng, Haibin Lin, Wencong Xiao, Xin Liu, and Liang Xiang. 2025. Robust LLM Training Infrastructure at ByteDance. arXiv:2509.16293 [cs.LG] https://arxiv.org/abs/2509.16293

[33] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. 2023. GEMINI: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 364–381. doi:10.1145/3600006.3613145

[34] Xingda Wei, Zhuobin Huang, Tianle Sun, Yingyi Hao, Rong Chen, Mingcong Han, Jinyu Gu, and Haibo Chen. 2025. PhoenixOS: Concurrent OS-level GPU Checkpoint and Restore with Validated Speculation. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) *(SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 996–1013. doi:10.1145/3731569.3764813

[35] Jinyi Xia, Menghao Zhang, Jiaxun Huang, Yuezheng Liu, Xiaohe Hu, Xudong Liu, and Chunming Hu. 2025. Mnemosyne: Lightweight and Fast Error Recovery for LLM Training in a Just-In-Time Manner. In *Proceedings of the 9th Asia-Pacific Workshop on Networking (APNET '25)*. Association for Computing Machinery, New York, NY, USA, 157–163. doi:10.1145/3735358.3735372

[36] Jiale Xu, Rui Zhang, Cong Guo, Weiming Hu, Zihan Liu, Feiyang Wu, Yu Feng, Shixuan Sun, Changxu Shao, Yuhong Guo, Junping Zhao, Ke Zhang, Minyi Guo, and Jingwen Leng. 2024. vTensor: Flexible Virtual Tensor Management for Efficient LLM Serving. arXiv:2407.15309 [cs.DC] https://arxiv.org/abs/2407.15309

[37] Chenxuan Yao, Feifan Liu, Yuchong Hu, Zhengyu Liu, Xinjue Zheng, and Wenxiang Zhou. 2025. LowDiff: Efficient Frequent Checkpointing via Low-Cost Differential for High-Performance Distributed Training Systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. Association for Computing Machinery, New York, NY, USA, 1113–1126. doi:10.1145/3712285.3759891

[38] Keyao Zhang, Yiquan Chen, Zhuo Hu, Wenhai Lin, Jiexiong Xu, and Wenzhi Chen. 2025. GoCkpt: Gradient-Assisted Multi-Step overlapped Checkpointing for Efficient LLM Training. *ArXiv* abs/2511.07035 (2025). https://api.semanticscholar.org/CorpusID:282912209

[39] Songyu Zhang, Aaron Tam, Myungjin Lee, Shixiong Qi, and K. K. Ramakrishnan. 2026. Making MoE-based LLM Inference Resilient with Tarragon. arXiv:2601.01310 [cs.DC] https://arxiv.org/abs/2601.01310

[40] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. arXiv:2304.11277 [cs.DC] https://arxiv.org/abs/2304.11277

[41] Wenxin Zheng, Wenxiao Wang, Yun Zhang, Mingcong Han, Bin Xu, Jinyu Gu, Xingda Wei, Haibo Chen, Zuquan Song, Gaohong Liu, Yucheng Nie, Zhe Nan, Zhuolin Zheng, Huan Yu, Shuguang Wang, Ziming Zhou, Hang Zhu, Wencong Xiao, and Xin Liu. 2026. SDCs in the Wild: Characterizing and Diagnosing SDC-Defective GPUs in Production LLM Training (Operational Systems). In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 1349–1367. https://www.usenix.org/conference/osdi26/presentation/zheng